

ЛАБОРАТОРНАЯ РАБОТА №1 (2 часа)

ИНТЕРПРЕТАТОР КОМАНДНОЙ СТРОКИ ОС MS WINDOWS

Цель работы – знакомство с возможностями интерпретатора командной строки и командами MS Windows

ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

1 Эволюция инструментов для автоматизации работы в ОС Microsoft Windows

В настоящее время графический интерфейс Windows стал настолько привычным, что многие пользователи и начинающие администраторы даже не задумываются об альтернативных способах управления данной ОС, связанных с командной строкой (command line) и различными сценариями (scripts), о тех преимуществах, которые дают эти инструменты с точки зрения автоматизации работы. Подобная ситуация обусловлена тем, что исторически командная строка всегда была слабым местом операционной системы Windows (по сравнению с Unix-системами).

При этом, однако, неправильно было бы думать, что командная строка или сценарии нужны только администраторам. Ведь ежедневные рутинные задачи пользователей (связанные, например, с копированием или архивированием файлов, подключением или отключением сетевых ресурсов и т.п.), которые обычно выполняются с помощью графического интерфейса проводника Windows, можно полностью самостоятельно автоматизировать, написав нехитрый командный файл, состоящий всего из нескольких строчек! Более того, для человека, не знающего основные команды Windows и такие базовые возможности ОС, как перенаправление ввода/вывода и конвейеризация команд, некоторые простейшие задачи могут показаться нетривиальными. Попробуйте, например, пользуясь только графическими средствами, сформировать файл, содержащий имена файлов из всех подкаталогов какого-либо каталога! А ведь для этого достаточно выполнить единственную команду DIR (с определенными ключами) и перенаправить вывод этой команды в нужный текстовый файл.

Необходимо, чтобы было реализовано следующее:

- работа в разных версиях ОС без установки какого-либо дополнительного программного обеспечения;
- интеграция с командной строкой (непосредственное выполнение вводимых с клавиатуры команд);
- согласованный и непротиворечивый синтаксис команд и утилит;
- наличие подробной встроенной справки по командам с примерами использования.

В ОС Windows дело обстоит сложнее. На сегодняшний день одного "идеального" средства автоматизации, удовлетворяющего сразу всем перечисленным выше требованиям, в Windows нет; в последних версиях ОС поддерживаются несколько стандартных инструментов автоматизации, сильно

отличающихся друг от друга: оболочка командной строки `cmd.exe`, среда выполнения сценариев Windows Script Host и оболочка Microsoft PowerShell. Поэтому администратору или пользователю Windows приходится выбирать, каким именно подходом воспользоваться для решения определенной задачи, а для этого желательно иметь четкое представление о сильных и слабых сторонах данных средств автоматизации. Рассмотрим достоинства и недостатки каждого из них.

1.1 Оболочка (интерпретатор) командной строки `command.com/cmd.exe`

Во всех версиях ОС Windows поддерживается интерактивная оболочка командной строки (command shell) и определенный набор утилит командной строки (количество и состав этих утилит зависит от версии ОС).

Механизм работы оболочек командной строки в разных системах одинаков: в ответ на приглашение ("подсказку", `prompt`), выдаваемое находящейся в ожидании оболочкой, пользователь вводит некоторую команду (функциональность этой команды может быть реализована либо самой оболочкой, либо определенной внешней утилитой), оболочка выполняет ее, при необходимости выводя на экран какую-либо информацию, после чего снова выводит приглашение и ожидает ввода следующей команды.

Оболочка представляет собой построчный интерпретатор простого языка сентенциального (директивного) программирования, в качестве операторов которого могут использоваться исполняемые программы.

Наряду с интерактивным режимом работы оболочки, как правило, поддерживают и пакетный режим, в котором система последовательно выполняет команды, записанные в текстовом файле-сценарии. Оболочка Windows не является исключением, с точки зрения программирования язык командных файлов Windows может быть охарактеризован следующим образом:

- реализация сентенциальной (директивной) парадигмы программирования;
- выполнение в режиме построчной интерпретации;
- наличие управляющих конструкций;
- поддержка нескольких видов циклов (в том числе специальных циклов для обработки текстовых файлов);
- наличие оператора присваивания (установки значения переменной);
- возможность использования внешних программ (команд) операционной системы в качестве операторов и обработки их кодов возврата;
- наличие нетипизированных переменных, которые декларируются первым упоминанием (значения переменных могут интерпретироваться как числа и использоваться в выражениях целочисленной арифметики).

Оболочка командной строки представляется интерпретатором `Cmd.exe`.

Итак, учитывая сказанное выше, можно сделать вывод: оболочка командной строки `cmd.exe` и командные файлы – наиболее универсальные и простые в

изучении средства автоматизации работы в Windows, доступные во всех версиях операционной системы.

1.2 Поддержка языков сценариев. Сервер сценариев Windows Script

Следующим шагом в развитии средств и технологий автоматизации в ОС Windows стало появление сервера сценариев Windows Script Host (WSH). Этот инструмент разработан для всех версий Windows и позволяет непосредственно в ОС выполнять сценарии на полноценных языках сценариев (по умолчанию, VBScript и JScript), которые до этого были доступны только внутри HTML-страниц и работали в контексте безопасности веб-браузера (в силу этого подобные сценарии, например, могли не иметь доступа к файловой системе локального компьютера).

По сравнению с командными файлами интерпретатора cmd.exe сценарии WSH имеют несколько преимуществ.

Во-первых, VBScript и JScript – это полноценные алгоритмические языки, имеющие встроенные функции и методы для обработки символьных строк, выполнения математических операций, обработки исключительных ситуаций и т.д.; кроме того, для написания сценариев WSH может использоваться любой другой язык сценариев (например, широко распространенный в Unix-системах Perl), для которого установлен соответствующий модуль поддержки.

Во-вторых, WSH поддерживает несколько собственных объектов, свойства и методы которых позволяют решать некоторые часто возникающие повседневные задачи администратора операционной системы: работа с сетевыми ресурсами, переменными среды, системным реестром, ярлыками и специальными папками Windows, запуск и управление работой других приложений.

В-третьих, из сценариев WSH можно обращаться к службам любых приложений-серверов автоматизации (например, программ из пакета MS Office), которые регистрируют в ОС свои объекты.

Наконец, сценарии WSH позволяют работать с объектами информационной модели Windows Management Instrumentation (WMI), обеспечивающей программный интерфейс управления всеми компонентами операционной модели, а также с объектами службы каталогов Active Directory Service Interface.

Следует также отметить, что технология WSH поддерживается в Windows уже довольно давно, в Интернете (в том числе на сайте Microsoft) можно найти множество готовых сценариев.

1.3 Командная оболочка Microsoft PowerShell

С одной стороны функциональности и гибкости языка оболочки cmd.exe явно недостаточно, а с другой стороны сценарии WSH, работающие с объектными моделями ADSI и WMI, слишком сложны для пользователей среднего уровня и начинающих администраторов.

Перед разработчиками новой оболочки, получившей название Windows PowerShell, стояли следующие основные цели:

-применение командной строки в качестве основного интерфейса администрирования;

-реализация модели ObjectFlow (элементом обмена информации является объект);

-переработка существующих команд, утилит и оболочки;

-интеграция командной строки, объектов COM, WMI и .NET;

-работа с произвольными источниками данных в командной строке по принципу файловой системы.

Самая важная идея, заложенная в PowerShell, состоит в том, что в командной строке вывод результатов команды представляет собой не текст (в смысле последовательности символов), а объект (данные вместе со свойствами и методами). В силу этого работать в PowerShell становится проще, чем в традиционных оболочках, так как не нужно выполнять никаких манипуляций по выделению нужной информации из символьного потока.

Отметим, что PowerShell одновременно является и оболочкой командной строки (пользователь работает в интерактивном режиме) и средой выполнения сценариев, которые пишутся на специальном языке PowerShell.

В целом, оболочка PowerShell намного удобнее и мощнее своих предшественников (cmd.exe и WSH).

2 Оболочка командной строки Windows. Интерпретатор Cmd.exe

Рассматриваются внутренние команды, поддерживаемые интерпретатором Cmd.exe, и наиболее часто используемые внешние команды (утилиты командной строки). Описываются механизмы перенаправления ввода/вывода, конвейеризации и условного выполнения команд.

В ОС Windows, как и в других ОС, интерактивные (набираемые с клавиатуры и сразу же выполняемые) команды выполняются с помощью так называемого командного интерпретатора, иначе называемого командным процессором или оболочкой командной строки (command shell). Начиная с версии Windows NT, в операционной системе реализован интерпретатор команд Cmd.exe, обладающий гораздо более широкими возможностями.

2.1 Запуск оболочки

В Windows файл Cmd.exe, как и другие исполняемые файлы, соответствующие внешним командам ОС, находятся в каталоге %SystemRoot%\SYSTEM32 (значением переменной среды %SystemRoot% является системный каталог Windows, обычно C:\Windows или C:\WinNT). Для запуска командного интерпретатора (открытия нового сеанса командной строки) можно выбрать пункт Выполнить... (Run) в меню Пуск (Start), ввести имя файла Cmd.exe и нажать кнопку ОК. В результате откроется новое окно, в котором можно запускать команды и видеть результат их работы.

2.2 Настройка параметров командного окна интерпретатора

У утилиты командной строки, которая поставляется в виде стандартного приложения ОС Windows, имеется свой набор опций и параметров настройки. Один из способов просмотра этих опций – использование пункта Свойства управляющего меню окна (нажать правой кнопкой мыши на заголовок окна). В окне свойств будут доступны четыре вкладки с опциями: общие, шрифт, расположение и цвета.

2.3 Внутренние и внешние команды. Структура команд

Некоторые команды распознаются и выполняются непосредственно самим командным интерпретатором — такие команды называются **внутренними** (например, COPY или DIR). Другие команды ОС представляют собой отдельные программы, расположенные по умолчанию в том же каталоге, что и Cmd.exe, которые Windows загружает и выполняет аналогично другим программам. Такие команды называются **внешними** (например, MORE или XCOPY).

Рассмотрим структуру самой командной строки и принцип работы с ней. Для того, чтобы выполнить команду, после приглашения командной строки (например, C:\>) следует ввести имя этой команды (регистр не важен), ее параметры и ключи (если они необходимы) и нажать клавишу <Enter>. Например:

```
C:\>COPY C:\myfile.txt A:\ /V
```

Имя команды здесь — COPY, параметры — C:\myfile.txt и A:\, а ключом является /V. Отметим, что в некоторых командах ключи могут начинаться не с символа /, а с символа – (минус), например, -V.

Многие команды Windows имеют большое количество дополнительных параметров и ключей, запомнить которые зачастую бывает трудно. Большинство команд снабжено встроенной справкой, в которой кратко описываются назначение и синтаксис данной команды. Получить доступ к такой справке можно путем ввода команды с ключом /?. Например, если выполнить команду ATTRIB /?, то в окне MS-DOS мы увидим следующий текст: Отображение и изменение атрибутов файлов.

```
ATTRIB [+R|-R] [+A|-A] [+S|-S] [+H|-H] [[диск:][путь]имя_файла] [/S]
```

+ Установка атрибута.

- Снятие атрибута.

R Атрибут "Только чтение".

A Атрибут "Архивный".

S Атрибут "Системный".

H Атрибут "Скрытый".

/S Обработка файлов во всех вложенных папках указанного пути.

Для некоторых команд текст встроенной справки может быть довольно большим и не уместиться на одном экране. В этом случае помощь можно выводить последовательно по одному экрану с помощью команды MORE и символа конвейеризации |, например:

```
XCOPY /? | MORE
```

В этом случае после заполнения очередного экрана вывод помощи будет прерываться до нажатия любой клавиши. Кроме того, используя символы перенаправления вывода > и >>, можно текст, выводимый на экран, направить в текстовый файл для дальнейшего просмотра. Например, для вывода текста справки к команде XCOPY в текстовый файл xcopy.txt, используется следующая команда:

XCOPY /? > XCOPY.TXT

Замечание. Вместо имени файла можно указывать обозначения устройств компьютера. В Windows поддерживаются следующие **имена устройств**: PRN (принтер), CON (терминал: при вводе это клавиатура, при выводе - монитор), NUL (пустое устройство, все операции ввода/вывода для него игнорируются).

2.4 Перенаправление ввода/вывода и конвейеризация (композиция) команд

С помощью переназначения устройств ввода/вывода одна программа может направить свой вывод на вход другой или перехватить вывод другой программы, используя его в качестве своих входных данных. Таким образом, имеется возможность передавать информацию от процесса к процессу при минимальных программных издержках. Практически это означает, что для программ, которые используют стандартные входные и выходные устройства, ОС позволяет:

- выводить сообщения программ не на экран (стандартный выходной поток), а в файл или на принтер (перенаправление вывода);

- читать входные данные не с клавиатуры (стандартный входной поток), а из заранее подготовленного файла (перенаправление ввода);

- передавать сообщения, выводимые одной программой, в качестве входных данных для другой программы (конвейеризация или композиция команд).

Из командной строки эти возможности реализуются следующим образом. Для того, чтобы перенаправить текстовые сообщения, выводимые какой-либо командой, в текстовый файл, нужно использовать конструкцию

команда > имя_файла

Если при этом заданный для вывода файл уже существовал, то он перезаписывается, если не существовал — создается. Можно также не создавать файл заново, а дописывать информацию, выводимую командой, в конец существующего файла. Для этого команда перенаправления вывода должна быть задана так:

команда >> имя_файла

С помощью символа < можно прочитать входные данные для заданной команды не с клавиатуры, а из определенного (заранее подготовленного) файла:

команда < имя_файла

Примеры команд перенаправления ввода/вывода:

Вывод встроенной справки для команды COPY в файл copy.txt:

COPY /? > copy.txt

Добавление текста справки для команды XCOPY в файл copy.txt:

XCOPY /? >> copy.txt

Вывод текущей даты в файл date.txt (DATE /T — это команда для просмотра и изменения системной даты, T ключ для получения только даты без запроса нового значения):

DATE /T > date.txt

Если при выполнении определенной команды возникает ошибка, то сообщение об этом по умолчанию выводится на экран. В случае необходимости сообщения об ошибках (стандартный поток ошибок) можно перенаправить в текстовый файл с помощью конструкции

команда 2> имя_файла

В этом случае стандартный вывод будет производиться на экран.

Также имеется возможность информировать сообщения и сообщения об ошибках выводить в один и тот же файл. Делается это следующим образом:

команда > имя_файла 2>&1

Например, в приведенной ниже команде стандартный выходной поток и стандартный поток ошибок перенаправляются в файл copy.txt:

XCOPY A:\1.txt C: > copy.txt 2>&1

Наконец, с помощью конструкции

команда1 | команда2 можно использовать сообщения, выводимые первой командой, в качестве входных данных для второй команды (конвейер команд).

Используя механизмы перенаправления ввода/вывода и конвейеризации, можно из командной строки посылать информацию на различные устройства и автоматизировать ответы на запросы, выдаваемые командами или программами, использующими стандартный ввод. Для решения таких задач служит команда

ECHO [сообщение], которая выводит сообщение на экран. Пример использования этой команды.

Удаление всех файлов в текущем каталоге без предупреждения (автоматический положительный ответ на запрос об удалении):

ECHO y | DEL *.*

2.5 Команды MORE и SORT

Одной из наиболее часто используемых команд, для работы с которой применяется перенаправление ввода/вывода и конвейеризация, является MORE. Эта команда считывает стандартный ввод из конвейера или перенаправленного файла и выводит информацию частями, размер каждой из которых не больше размера экрана. Используется MORE обычно для просмотра длинных файлов. Возможны три варианта синтаксиса этой команды:

MORE [диск:][путь]имя_файла

MORE < [диск:][путь]имя_файла

имя_команды | MORE

Параметр [диск:] [путь] имя_файла определяет расположение и имя файла с просматриваемыми на экране данными. Параметр имя_команды задает команду, вывод которой отображается на экране (например, DIR или команда

TYPE, использующаяся для вывода содержимого текстового файла на экран). Приведем два примера.

Для поэкранного просмотра текстового файла news.txt возможны следующие варианты команд:

```
MORE news.txt
```

```
MORE < news.txt
```

```
TYPE news.txt | MORE 15
```

Другой распространенной командой, использующей перенаправление ввода/вывода и конвейеризацию, является SORT. Эта команда работает как фильтр: она считывает символы в заданном столбце, упорядочивает их в возрастающем или убывающем порядке и выводит отсортированную информацию в файл, на экран или другое устройство. Возможны два варианта синтаксиса этой команды:

```
SORT [/R] [/+n] [[диск1:][путь1]файл1] [> [диск2:][путь2]файл2]
```

или

```
[команда |] SORT [/R] [/+n] [> [диск2:][путь2]файл2]
```

В первом случае параметр [диск1:] [путь1] файл1 определяет имя файла, который нужно отсортировать. Во втором случае будут отсортированы выходные данные указанной команды. Если параметры файл1 или команда не заданы, то SORT будет считывать данные с устройства стандартного ввода.

Параметр [диск2:] [путь2] файл2 задает файл, в который будет направляться сортированный вывод; если этот параметр не задан, то вывод будет направлен на устройство стандартного вывода.

По умолчанию сортировка выполняется в порядке возрастания. Ключ /R позволяет изменить порядок сортировки на обратный (от Z к A и затем от 9 до 0). Например, для поэкранного просмотра отсортированного в обратном порядке файла price.txt, нужно задать следующую команду:

```
SORT /R < price.txt |MORE
```

Ключ /+n задает сортировку в файле по символам n-го столбца. Например, /+10 означает, что сортировка должна осуществляться, начиная с 10-й позиции в каждой строке. По умолчанию файл сортируется по первому столбцу.

2.6 Условное выполнение и группировка команд

В командной строке Windows можно использовать специальные символы, которые позволяют вводить несколько команд одновременно и управлять работой команд в зависимости от результатов их выполнения. С помощью таких символов условной обработки можно содержание небольшого пакетного файла записать в одной строке и выполнить полученную составную команду.

Используя символ амперсанда &, можно разделить несколько утилит в одной командной строке, при этом они будут выполняться друг за другом. Например, если набрать команду DIR & PAUSE & COPY /? и нажать клавишу <Enter>, то вначале на экран будет выведено содержимое текущего каталога, а после нажатия любой клавиши — встроенная справка команды COPY.

Условная обработка команд в Windows осуществляется с помощью символов `&&` и `||` следующим образом. Двойной амперсанд `&&` запускает команду, стоящую за ним в командной строке, только в том случае, если команда, стоящая перед амперсандами была выполнена успешно. Например, если в корневом каталоге диска C: есть файл `plan.txt`, то выполнение строки `TYPE C:\plan.txt && DIR` приведет к выводу на экран этого файла и содержимого текущего каталога. Если же файл `C:\plan.txt` не существует, то команда `DIR` выполняться не будет.

Два символа `||` осуществляют в командной строке обратное действие, т.е. запускают команду, стоящую за этими символами, только в том случае, если команда, идущая перед ними, не была успешно выполнена. Таким образом, если в предыдущем примере файл `C:\plan.txt` будет отсутствовать, то в результате выполнения строки `TYPE C:\plan.txt || DIR` на экран выведется содержимое текущего каталога.

Отметим, что условная обработка действует только на ближайшую команду, то есть в строке

```
TYPE C:\plan.txt && DIR & COPY /?
```

команда `COPY /?` запустится в любом случае, независимо от результата выполнения команды `TYPE C:\plan.txt`.

Несколько утилит можно сгруппировать в командной строке с помощью *круглых скобок*.

Пример:

```
TYPE C:\plan.txt && DIR & COPY /?
```

```
TYPE C:\plan.txt && (DIR & COPY /?)
```

В первой из них символ условной обработки `&&` действует только на команду `DIR`, во второй — одновременно на две команды: `DIR` и `COPY`.

3 Команды для работы с файловой системой

Рассмотрим некоторые наиболее часто используемые команды для работы с файловой системой. Отметим сначала несколько особенностей определения путей к файлам в Windows.

Файловая система логически имеет древовидную структуру и имена файлов задаются в формате `[диск:][путь\]имя_файла`, то есть обязательным параметром является только имя файла. При этом, если путь начинается с символа `"\"`, то маршрут вычисляется от корневого каталога, иначе — от текущего каталога. Например, имя `C:123.txt` задает файл `123.txt` в текущем каталоге на диске C:, имя `C:\123.txt` — файл `123.txt` в корневом каталоге на диске C:, имя `ABC\123.txt` — файл `123.txt` в подкаталоге `ABC` текущего каталога.

Существуют особые обозначения для текущего каталога и родительского каталогов. Текущий каталог обозначается символом `.` (точка), его родительский каталог — символами `..` (две точки). Например, если текущим каталогом является `C:\WINDOWS`, то путь к файлу `autoexec.bat` в корневом каталоге диска C: может быть записан в виде `..\autoexec.bat`.

В именах файлов (но не дисков или каталогов) можно применять так называемые **групповые символы** или шаблоны: ? (вопросительный знак) и * (звездочка). Символ * в имени файла означает произвольное количество любых допустимых символов, символ ? — один произвольный символ или его отсутствие. Скажем, под шаблон text??1.txt подходят, например, имена text121.txt и text11.txt, под шаблон text*.txt — имена text.txt, textab12.txt, а под шаблон text.* — все файлы с именем text и произвольным расширением.

Для того, чтобы использовать длинные имена файлов при работе с командной строкой, их нужно заключать в двойные кавычки. Например, чтобы запустить файл с именем 'Мое приложение.exe' из каталога 'Мои документы', нужно в командной строке набрать "C:\Мои документы\Мое приложение.exe" и нажать клавишу <Enter>.

Текущий каталог можно изменить с помощью команды CD [диск:][путь\]. Путь к требуемому каталогу указывается с учетом приведенных выше замечаний. Например, команда CD \ выполняет переход в корневой каталог текущего диска. Если запустить CD без параметров, то на экран будут выведены имена текущего диска и каталога.

Одной из наиболее часто повторяющихся задач при работе на компьютере является копирование и перемещение файлов из одного места в другое. Для копирования одного или нескольких файлов используется команда COPY.

Синтаксис этой команды:

COPY [/A/B] источник [/A/B] [+ источник [/A/B] [+ ...]]
[результат [/A/B]] [/V][/Y|/Y]

Краткое описание параметров и ключей команды COPY приведено в (табл. 1).

Примеры использования команды COPY.

Копирование файла abc.txt из текущего каталога в каталог D:\PROGRAM под тем же именем:

COPY abc.txt D:\PROGRAM

Копирование файла abc.txt из текущего каталога в каталог D:\PROGRAM под новым именем def.txt:

COPY abc.txt D:\PROGRAM\def.txt

Копирование всех файлов с расширением txt с диска A: в каталог 'Мои документы' на диске C:

COPY A:*.txt "C:\Мои документы"

Если не задать в команде целевой файл, то команда COPY создаст копию файла-источника с тем же именем, датой и временем создания, что и исходный файл, и поместит новую копию в текущий каталог на текущем диске. Например, для того, чтобы скопировать все файлы из корневого каталога диска A: в текущий каталог, достаточно выполнить такую команду:

COPY A:*.*

Таблица 1.

Параметры и ключи команды COPY

Параметр	Описание
источник	Имя копируемого файла или файлов
/A	Файл является текстовым файлом ASCII, то есть конец файла обозначается символом с кодом ASCII 26 (<Ctrl>+<Z>)
/B	Файл является двоичным. Этот ключ указывает на то, что интерпретатор команд должен при копировании считывать из источника число байт, заданное размером в каталоге копируемого файла
результат	Каталог для размещения результата копирования и/или имя создаваемого файла
/V	Проверка правильности копирования путем сравнения файлов после копирования
/Y	Отключение режима запроса подтверждения на замену файлов
/-Y	Включение режима запроса подтверждения на замену файлов

Пример 1. Создания нового текстового файла и записи в него информации без использования текстового редактора.

Для решения задачи необходимо ввести команду `COPY CON my.txt`, которая будет копировать то, что набирается на клавиатуре в файл `my.txt` (если этот файл существовал, то он перезапишется, иначе — создастся). Для завершения ввода необходимо ввести символ конца файла, то есть нажать клавиши <Ctrl>+<Z>.

Команда `COPY` может также объединять (склеивать) несколько файлов в один. Для этого необходимо указать единственный результирующий файл и несколько исходных. Это достигается путем использования групповых знаков (? и *) или формата `файл1 + файл2 + файл3`. Например, для объединения файлов `1.txt` и `2.txt` в файл `3.txt` можно задать следующую команду:

`COPY 1.txt+2.txt 3.txt`

Объединение всех файлов с расширением `dat` из текущего каталога в один файл `all.dat` может быть произведено так:

`COPY /B *.dat all.dat`

Ключ `/B` здесь используется для предотвращения усечения соединяемых файлов, так как при комбинировании файлов команда `COPY` по умолчанию считает файлами текстовыми.

Если имя целевого файла совпадает с именем одного из копируемых файлов (кроме первого), то исходное содержимое целевого файла теряется. Если имя целевого файла опущено, то в его качестве используется первый файл из списка. Например, команда `COPY 1.txt+2.txt` добавит к содержимому файла `1.txt` содержимое файла `2.txt`. Командой `COPY` можно воспользоваться и для

присвоения какому-либо файлу **текущей даты и времени** без модификации его содержимого. Для этого нужно ввести команду

`COPY /B 1.txt +,,`

Здесь запятые указывают на пропуск параметра приемника, что и приводит к требуемому результату.

Команда `COPY` имеет и свои недостатки. Например, с ее помощью нельзя копировать скрытые и системные файлы, файлы нулевой длины, файлы из подкаталогов. Кроме того, если при копировании группы файлов `COPY` встретит файл, который в данный момент нельзя скопировать (например, он занят другим приложением), то процесс копирования полностью прервется, и остальные файлы не будут скопированы.

Указанные при описании команды `COPY` проблемы можно решить с помощью команды `XCOPY`, которая предоставляет намного больше возможностей при копировании. Необходимо отметить, правда, что `XCOPY` может работать только с файлами и каталогами, но не с **устройствами**.

Синтаксис команды: `XCOPY` источник [результат] [ключи]

Команда `XCOPY` имеет множество ключей, далее приведены лишь некоторых из них. Ключ `/D[:[дата]]` позволяет копировать только файлы, измененные не ранее указанной даты. Если параметр дата не указан, то копирование будет производиться только если источник новее результата. Например, команда `XCOPY "C:\Мои документы\*.*" "D:\BACKUP\Мои документы" /D` скопирует в каталог 'D:\BACKUP\Мои документы' только те файлы из каталога 'C:\Мои документы', которые были изменены со времени последнего подобного копирования или которых вообще не было в 'D:\BACKUP\Мои документы'.

Ключ `/S` позволяет копировать все непустые подкаталоги в каталоге-источнике. С помощью же ключа `/E` можно копировать вообще все подкаталоги, включая и пустые.

Если указан ключ `/C`, то копирование будет продолжаться даже в случае возникновения ошибок. Это бывает очень полезным при операциях копирования, производимых над группами файлов, например, при резервном копировании данных.

Ключ `/I` важен для случая, когда копируются несколько файлов, а файл назначения отсутствует. При задании этого ключа команда `XCOPY` считает, что файл назначения должен быть каталогом. Например, если задать ключ `/I` в команде копирования всех файлов с расширением `txt` из текущего каталога в несуществующий еще подкаталог `TEXT`, `XCOPY *.txt TEXT /I` то подкаталог `TEXT` будет создан без дополнительных запросов.

Ключи `/Q`, `/F` и `/L` отвечают за режим отображения при копировании. При задании ключа `/Q` имена файлов при копировании не отображаются, ключа `/F` — отображаются полные пути источника и результата. Ключ `/L` обозначает, что отображаются только файлы, которые должны быть скопированы (при этом само копирование не производится).

С помощью ключа /H можно копировать скрытые и системные файлы, а с помощью ключа /R — заменять файлы с атрибутом "Только для чтения". Например, для копирования всех файлов из корневого каталога диска C: (включая системные и скрытые) в каталог SYS на диске D:, нужно ввести следующую команду:

```
XCOPY C:\*.* D:\SYS /H
```

Ключ /T позволяет применять XCOPY для копирования только структуры каталогов источника, без дублирования находящихся в этих каталогах файлов, причем пустые каталоги и подкаталоги не включаются. Для того, чтобы все же включить пустые каталоги и подкаталоги, нужно использовать комбинацию ключей /T /E.

Используя XCOPY можно при копировании обновлять только уже существующие файлы (новые файлы при этом не записываются). Для этого применяется ключ /U. Например, если в каталоге C:\2 находились файлы a.txt и b.txt, а в каталоге C:\1 — файлы a.txt, b.txt, c.txt и d.txt, то после выполнения команды:

```
XCOPY C:\1 C:\2 /U
```

в каталоге C:\2 по-прежнему останутся лишь два файла a.txt и b.txt, содержимое которых будет заменено содержимым соответствующих файлов из каталога C:\1. Если с помощью XCOPY копировался файл с атрибутом "Только для чтения", то по умолчанию у файла-копии этот атрибут снимется. Для того, чтобы копировать не только данные, но и полностью атрибуты файла, необходимо использовать ключ /K.

Ключи /Y и /-Y определяют, нужно ли запрашивать подтверждение перед заменой файлов при копировании. /Y означает, что такой запрос нужен, /-Y — не нужен.

Команда: DIR [диск:] [путь] [имя_файла] [ключи] используется для вывода информации о содержимом дисков и каталогов. Параметр [диск:] [путь] задает диск и каталог, содержимое которого нужно вывести на экран. Параметр [имя_файла] задает файл или группу файлов, которые нужно включить в список.

Например, команда DIR C:*.bat выведет на экран все файлы с расширением bat в корневом каталоге диска C:. Если задать эту команду без параметров, то выводится метка диска и его серийный номер, имена (в коротком и длинном вариантах) файлов и подкаталогов, находящихся в текущем каталоге, а также дата и время их последней модификации. После этого выводится число файлов в каталоге, общий объем (в байтах), занимаемый файлами, и объем свободного пространства на диске. Например: Том в устройстве C имеет метку PHYS1_PART2

Серийный номер тома: 366D-6107

Содержимое папки C:\aditor

. <ПАПКА> 25.01.15 17:15 .

.. <ПАПКА> 25.01.15 17:15 ..

```
HILITE DAT 1 082 18.09.16 18:55 hilite.dat
TEMPLT01 DAT 48 07.08.16 1:00 templt01.dat
TTABLE DAT 357 07.08.16 1:00 ttable.dat
ADITOR EXE 461 312 01.12.15 23:13 aditor.exe
README TXT 3 974 25.01.15 17:26 readme.txt
ADITOR HLP 24 594 08.10.16 23:12 aditor.hlp
ТЕКСТО~1 TXT 0 11.03.15 9:02 Текстовый файл.txt
11 файлов 533 647 байт
2 папок 143 261 696 байт свободно
```

С помощью ключей команды DIR можно задать различные режимы расположения, фильтрации и сортировки. Например, при использовании ключа /W перечень файлов выводится в широком формате с максимально возможным числом имен файлов или каталогов на каждой строке. Например: Том в устройстве С имеет метку PHYS1_PART2

```
Серийный номер тома: 366D-6107
Содержимое папки C:\aditor
[.] [..] TEMPLT02.DAT UNINST1.000 HILITE.DAT
TEMPLT01.DAT UNINST0.000 TTABLE.DAT ADITOR.EXE README.TXT
ADITOR.HLP ТЕКСТО~1.TXT
```

```
11 файлов 533 647 байт
2 папок 143 257 600 байт свободно
```

С помощью ключа /A [[:] атрибуты] можно вывести имена только тех каталогов и файлов, которые имеют заданные атрибуты (R — "Только чтение", A — "Архивный", S — "Системный", H — "Скрытый", префикс "-" имеет значение NE). Если ключ /A используется более чем с одним значением атрибута, будут выведены имена только тех файлов, у которых все атрибуты совпадают с заданными. Например, для вывода имен всех файлов в корневом каталоге диска C:, которые одновременно являются скрытыми и системными, можно задать команду

```
DIR C:\ /A:HS
```

а для вывода всех файлов, кроме скрытых — команду

```
DIR C:\ /A:-H
```

Отметим здесь, что атрибуту каталога соответствует буква D, и для того, чтобы, например, вывести список всех каталогов диска C:, нужно задать команду

```
DIR C: /A:D
```

Ключ /O [[:] сортировка] задает порядок сортировки содержимого каталога при выводе его командой DIR. Если этот ключ опущен, DIR печатает имена файлов и каталогов в том порядке, в котором они содержатся в каталоге. Если ключ /O задан, а параметр сортировка не указан, то DIR выводит имена в алфавитном порядке. В параметре сортировка можно использовать следующие значения: N — по имени (алфавитная), S — по размеру (начиная с меньших), E — по расширению (алфавитная), D — по дате (начиная с более старых), A — по дате загрузки (начиная с более старых), G — начать список с каталогов. Префикс

"—" означает обратный порядок. Если задается более одного значения порядка сортировки, файлы сортируются по первому критерию, затем по второму и т.д.

Ключ /S означает вывод списка файлов из заданного каталога и его подкаталогов. Ключ /B перечисляет только названия каталогов и имена файлов (в длинном формате) по одному на строку, включая расширение. При этом выводится только основная информация, без итоговой. Например:

```
templt02.dat
UNINST1.000
hilite.dat
templt01.dat
UNINST0.000
ttable.dat
aditor.exe
readme.txt
aditor.hlp
Текстовый файл.txt
```

Для создания нового каталога и удаления уже существующего пустого каталога используются команды MKDIR [диск:]путь и RMDIR [диск:]путь [ключи] соответственно (или их короткие аналоги MD и RD). Например:

```
MKDIR "C:\Примеры"
RMDIR "C:\Примеры"
```

Команда MKDIR не может быть выполнена, если каталог или файл с заданным именем уже существует. Команда RMDIR не будет выполнена, если удаляемый каталог не пустой.

Удалить один или несколько файлов можно с помощью команды
DEL [диск:][путь]имя_файла [ключи]

Для удаления сразу нескольких файлов используются групповые знаки ? и *. Ключ /S позволяет удалить указанные файлы из всех подкаталогов, ключ /F – принудительно удалить файлы, доступные только для чтения, ключ /A[:]атрибуты – отбирать файлы для удаления по атрибутам (аналогично ключу /A[:]атрибуты в команде DIR).

Переименовать файлы и каталоги можно с помощью команды RENAME (REN).

Синтаксис этой команды имеет следующий вид:

```
REN [диск:][путь][каталог1|файл1] [каталог2|файл2]
```

Здесь параметр каталог1|файл1 определяет название каталога/файла, которое нужно изменить, а каталог2|файл2 задает новое название каталога/файла. В любом параметре команды REN можно использовать групповые символы ? и *. При этом представленные шаблонами символы в параметре файл2 будут идентичны соответствующим символам в параметре файл1. Например, чтобы изменить у всех файлов с расширением txt в текущей директории расширение на doc, нужно ввести такую команду:

```
REN *.txt *.doc
```

Если файл с именем файл2 уже существует, то команда REN прекратит выполнение, и произойдет вывод сообщения, что файл уже существует или занят. Кроме того, в команде REN нельзя указать другой диск или каталог для создания результирующих каталога и файла. Для этой цели нужно использовать команду MOVE, предназначенную для переименования и перемещения файлов и каталогов.

Синтаксис команды для перемещения одного или более файлов имеет вид:

MOVE [/Y|/Y] [диск:][путь]имя_файла1[,...] результирующий_файл

Синтаксис команды для переименования папки имеет вид:

MOVE [/Y|/Y] [диск:][путь]каталог1 каталог2

Здесь параметр результирующий_файл задает новое размещение файла и может включать имя диска, двоеточие, имя каталога, либо их сочетание. Если перемещается только один файл, допускается указать его новое имя. Это позволяет сразу переместить и переименовать файл. Например, MOVE "C:\Мои документы\список.txt" D:\list.txt.

Если указан ключ /Y, то при создании каталогов и замене файлов будет выдаваться запрос на подтверждение. Ключ /Y отменяет выдачу такого запроса.

4 Язык интерпретатора Cmd.exe. Командные файлы

Язык оболочки командной строки (shell language) в Windows реализован в виде командных (или пакетных) файлов. **Командный файл** в Windows — это обычный текстовый файл с расширением bat или cmd, в котором записаны допустимые команды ОС (как внешние, так и внутренние), а также некоторые дополнительные инструкции и ключевые слова, придающие командным файлам некоторое сходство с программами, написанными на языке программирования. Например, если записать в файл deltmp.bat следующие команды:

```
C:\  
CD %TEMP%
```

DEL /F *.tmp и запустить его на выполнение (аналогично исполняемому файлам с расширением com или exe), то мы удалим все файлы во временной директории Windows. Таким образом, исполнение командного файла приводит к тому же результату, что и последовательный ввод записанных в нем команд. При этом не проводится никакой предварительной компиляции или проверки синтаксиса кода; если встречается строка с ошибочной командой, то она игнорируется. Очевидно, что если приходится часто выполнять одни и те же действия, то использование командных файлов может сэкономить много времени.

4.1 Вывод сообщений и дублирование команд

По умолчанию команды пакетного файла перед исполнением выводятся на экран, что выглядит не очень эстетично. С помощью команды ECHO OFF можно отключить дублирование команд, идущих после нее (сама команда ECHO OFF при этом все же дублируется). Например,

```
REM Следующие две команды будут дублироваться на экране ...  
:: эта строка – такой же комментарий, как и  
предыдущая DIR C:\  
ECHO OFF  
:: А остальные уже не будут  
DIR D:\
```

Для восстановления режима дублирования используется команда ECHO ON. Кроме этого, можно отключить дублирование любой отдельной строки в командном файле, написав в начале этой строки символ @, например:

```
ECHO ON  
:: Команда DIR C:\ дублируется на экране DIR C:\  
:: А команда DIR D:\ — нет @DIR D:\  
Таким образом, если поставить в самое начало файла команду @ECHO  
OFF, то это решит все проблемы с дублированием команд.
```

В пакетном файле можно выводить на экран строки с сообщениями.

Делается это с помощью команды ECHO сообщение Например:

```
@ECHO OFF  
ECHO Привет!  
Команда ECHO. (точка должна следовать непосредственно за словом  
"ECHO") выводит на экран пустую строку. Например:  
@ECHO OFF  
ECHO Привет!  
ECHO.  
ECHO Пока!
```

Часто бывает удобно для просмотра сообщений, выводимых из пакетного файла, предварительно полностью очистить экран командой CLS.

Используя механизм *перенаправления ввода/вывода* (символы > и >>), можно направить сообщения, выводимые командой

ECHO, в определенный текстовый файл. Например:

```
@ECHO OFF  
ECHO Привет! > hi.txt ECHO Пока! >> hi.txt
```

С помощью такого метода можно, скажем, заполнять файлы-протоколы с отчетом о произведенных действиях. Например:

```
@ECHO OFF  
REM Попытка копирования  
XCOPY C:\PROGRAMS D:\PROGRAMS /s  
:: Добавление сообщения в файл report.txt в случае удачного завершения  
копирования
```

IF NOT ERRORLEVEL 1 ECHO Успешное копирование >> report.txt

Использование параметров командной строки

При запуске пакетных файлов в командной строке можно указывать произвольное число параметров, значения которых можно использовать внутри файла. Это позволяет, например, применять один и тот же командный файл для выполнения команд с различными параметрами.

Для доступа из командного файла к параметрам командной строки применяются символы %0, %1, ..., %9 или %*. При этом вместо %0 подставляется имя выполняемого пакетного файла, вместо %1, %2, ..., %9

— значения первых девяти параметров командной строки соответственно, а вместо %* — все аргументы. Если в командной строке при вызове пакетного файла задано меньше девяти параметров, то "лишние" переменные из %1 – %9 замещаются пустыми строками. Рассмотрим следующий пример. Пусть имеется командный файл copier.bat следующего содержания:

```
@ECHO OFF CLS
```

```
ECHO Файл %0 копирует каталог %1 в %2
```

```
XCOPY %1 %2 /S
```

Если запустить его из командной строки с двумя параметрами, например copier.bat C:\Programs D:\Backup то на экран выведется сообщение

Файл copier.bat копирует каталог C:\Programs в D:\Backup

и произойдет копирование каталога

C:\Programs со всеми его подкаталогами в D:\Backup.

При необходимости можно использовать более девяти параметров командной строки. Это достигается с помощью команды **SHIFT**, которая изменяет значения замещаемых параметров с %0 по %9, копируя каждый параметр в предыдущий, то есть значение %1 копируется в %0, значение

%2 – в %1 и т.д. Замещаемому параметру %9 присваивается значение параметра, следующего в командной строке за старым значением %9. Если же такой параметр не задан, то новое значение %9 — пустая строка.

Пример 1. Пусть командный файл my.bat вызван из командной строки следующим образом:

```
my.bat p1 p2 p3
```

Тогда %0=my.bat, %1=p1, %2=p2, %3=p3, параметры %4 – %9 являются пустыми строками. После выполнения команды SHIFT значения замещаемых параметров изменятся следующим образом: %0=p1, %1=p2,

%2=p3, параметры %3 – %9 – пустые строки.

При включении расширенной обработки команд SHIFT поддерживает ключ /n, задающий начало сдвига параметров с номера n, где n может быть числом от 0 до 9.

Например, в следующей команде SHIFT /2 параметр %2 заменяется на %3, %3 на %4 и т.д., а параметры %0 и %1 остаются без изменений.

Команда, обратная SHIFT (обратный сдвиг), отсутствует. После выполнения SHIFT уже нельзя восстановить параметр (%0), который был первым

перед сдвигом. Если в командной строке задано больше десяти параметров, то команду SHIFT можно использовать несколько раз.

В командных файлах имеются некоторые возможности синтаксического анализа заменяемых параметров. Для параметра с номером *n* (%*n*) допустимы синтаксические конструкции (операторы), представленные в (табл. 2).

Таблица 2 - Операторы для заменяемых параметров

Операторы	Описание
%~Fn	Переменная % <i>n</i> расширяется до полного имени файла
%~Dn	Из переменной % <i>n</i> выделяется только имя диска
%~Pn	Из переменной % <i>n</i> выделяется только путь к файлу
%~Nn	Из переменной % <i>n</i> выделяется только имя файла
%~Xn	Из переменной % <i>n</i> выделяется расширение имени файла
%~Tn	Возвращается дата и время создания (модификации) файла
%~Zn	Возвращается размер файла в байтах
%~\$PATH:n	Проводится поиск по каталогам, заданным в переменной среды PATH, и переменная % <i>n</i> заменяется на полное имя первого найденного файла. Если переменная PATH не определена или в результате поиска не найден ни один файл, эта конструкция заменяется на пустую строку. Естественно, здесь переменную PATH можно заменить на любое другое допустимое значение

Данные синтаксические конструкции можно объединять друг с другом, например:

%~DPn — из переменной %*n* выделяется имя диска и путь,

%~NXn — из переменной %*n* выделяется имя файла и расширение.

Пример 2. Пусть мы находимся в каталоге C:\TEXT и запускаем пакетный файл с параметром Рассказ.doc (%1=Рассказ.doc). Размер файла 2150 байт, дата создания 12.12.2015, время -12:55. Тогда применение операторов, описанных в табл. 1, к параметру %1 даст следующие результаты:

%~F1=C:\TEXT\Рассказ.doc

%~D1=C:

%~P1=\TEXT\

%~N1=Рассказ

%~X1=.doc

%~DP1=C:\TEXT\

%~NX1=Рассказ.doc

%~T1=12.12.2009 12:55

%~Z1=2150

Работа с переменными среды

Внутри командных файлов можно использовать так называемые **переменными среды** (или переменными окружения), каждая из которых хранится в оперативной памяти, имеет свое уникальное имя, а ее значением является **строка**. Стандартные переменные среды автоматически инициализируются в процессе загрузки операционной системы. Такими переменными являются:

WINDIR, которая определяет расположение каталога Windows, TEMP, которая определяет путь к каталогу для хранения временных файлов Windows

- PATH, в которой хранится системный путь (путь поиска), то есть список каталогов, в которых система должна искать выполняемые файлы или файлы совместного доступа (например, динамические библиотеки).

Кроме того, в командных файлах с помощью команды SET можно объявлять собственные переменные среды.

Получение значения переменной

Для получения значения определенной переменной среды нужно заключить имя этой переменной в символы %. Например:

```
@ECHO OFF
```

```
CLS
```

```
:: Создание переменной MyVar SET MyVar=Привет
```

```
:: Изменение переменной SET MyVar=%MyVar%
```

```
ECHO Значение переменной MyVar: %MyVar%
```

```
:: Удаление переменной MyVar SET MyVar=
```

```
ECHO Значение переменной WinDir: %WinDir%
```

При запуске такого командного файла на экран выведется строка Значение переменной MyVar: Привет!

Значение переменной WinDir: C:\WINDOWS

Преобразования переменных как строк

С переменными среды в командных файлах можно производить некоторые манипуляции. Во-первых, над ними можно производить операцию конкатенации (соединения). Для этого нужно в команде SET просто написать рядом значения соединяемых переменных. Например, SET A=Раз

```
SET B=Два
```

```
SET C=%A%%B%
```

После выполнения в файле этих команд значением переменной C будет являться строка 'РазДва'. Не следует для конкатенации использовать знак +, так как он будет воспринят просто в качестве символа. Например, после запуска файл следующего содержания

```
SET A=Раз SET B=Два SET C=A+B
```

```
ECHO Переменная C=%C% SET D=%A%+%B%
```

```
ECHO Переменная D=%D%
```

на экран выведутся две строки:

Переменная C=A+B Переменная D=Раз+Два

Во-вторых, из переменной среды можно **выделять подстроки** с помощью конструкции `%имя_переменной:~n1,n2%`, где число `n1` определяет смещение (количество пропускаемых символов) от начала (если `n1` положительно) или от конца (если `n1` отрицательно) соответствующей переменной среды, а число `n2` – количество выделяемых символов (если `n2` положительно) или количество последних символов в переменной, которые не войдут в выделяемую подстроку (если `n2` отрицательно). Если указан только один отрицательный параметр `-n`, то будут извлечены последние `n` символов. Например, если в переменной хранится строка `"21.12.2015"` (символьное представление текущей даты при определенных региональных настройках), то после выполнения следующих команд

```
SET dd1=%DATE:~0,2% SET dd2=%DATE:~0,-8% SET mm=%DATE:~-7,2% SET yyyy=%DATE:~-4%
```

новые переменные будут иметь такие значения: `%dd1%=21`,
`%dd2%=21`,
`%mm%=12`, `%yyyy%=2015`.

В-третьих, можно выполнять процедуру замены подстрок с помощью конструкции `%имя_переменной:s1=s2%` (в результате будет возвращена строка, в которой каждое вхождение подстроки `s1` в соответствующей переменной среды заменено на `s2`). Например, после выполнения команд

```
SET a=123456 SET b=%a:23=99%
```

в переменной `b` будет храниться строка `"199456"`. Если параметр `s2` не указан, то подстрока `s1` будет удалена из выводимой строки, т.е. после выполнения команды

```
SET a=123456 SET b=%a:23=%
```

в переменной `b` будет храниться строка `"1456"`.

Операции с переменными как с числами

При включенной расширенной обработке команд (этот режим в Windows используется по умолчанию) имеется **возможность рассматривать значения переменных среды как числа** и производить с ними арифметические вычисления (используются ТОЛЬКО целые числа). Для этого используется команда `SET` с ключом `/A`. Ниже приведен пример пакетного файла `add.bat`, складывающего два числа, заданных в качестве параметров командной строки, и выводящего полученную сумму на экран: `@ECHO OFF`

```
:: В переменной M будет храниться сумма SET /A M=%1+%2
```

```
ECHO Сумма %1 и %2 равна %M%
```

```
:: Удалим переменную M SET M=
```

В команде `SET` с ключом `/A` могут использоваться операции – (вычитание), `*` (умножение), `/` (деление нацело), `%` (остаток от деления). При использовании знака `%` в качестве знака операции в **командных файлах** он должен быть записан **ДВАЖДЫ**.

Рекомендуется при инициализации числовых переменных использовать

ключ /A

SET /A col=0

Ввод значения переменной с клавиатуры

Ввод значения переменной при выполнении командного файла выполняется командой SET с ключом /P. Например, для ввода значения переменной M следует использовать команду

SET /P M=[введите M]

Текст подсказки [введите M] будет выведен на экран.

Локальные изменения переменных

Все изменения, производимые с помощью команды SET над переменными среды в командном файле, сохраняются и после завершения работы этого файла, но действуют только внутри текущего командного окна. Также имеется возможность локализовать изменения переменных среды внутри пакетного файла, то есть автоматически восстанавливать значения всех переменных в том виде, в каком они были до начала запуска этого файла. Для этого используются две команды: SETLOCAL и ENDLOCAL. Команда SETLOCAL определяет начало области локальных установок переменных среды. Другими словами, изменения среды, внесенные после выполнения SETLOCAL, будут являться локальными относительно текущего пакетного файла. Каждая команда SETLOCAL должна иметь соответствующую команду ENDLOCAL для восстановления прежних значений переменных среды. Изменения среды, внесенные после выполнения команды ENDLOCAL, уже не являются локальными относительно текущего пакетного файла; их прежние значения не будут восстановлены по завершении выполнения этого файла.

Связывание времени выполнения для переменных

При работе с составными выражениями (группы команд, заключенных в круглые скобки) нужно учитывать, что переменные среды в командных файлах используются в режиме раннего связывания. С точки зрения логики выполнения командного файла это может привести к ошибкам. Например, рассмотрим командный файл 1.bat со следующим содержимым:

SET a=1 ECHO a=%a% SET a=2 ECHO a=%a%

и командный файл 2.bat:

SET a=1 ECHO a=%a% (SET a=2
ECHO a=%a%)

Казалось бы, результат выполнения этих двух файлов должен быть одинаковым: на экран выведутся две строки: "a=1" и "a=2". На самом же деле таким образом сработает только файл 1.bat, а файл 2.bat два раза выведет строку "a=1".

Данную ошибку можно обойти, если для получения значения переменной вместо знаков процента (%) использовать восклицательный знак (!) и предварительно включить режим связывания времени

выполнения командой **SETLOCAL ENABLEDELAYEDEXPANSION**. Таким образом, для корректной работы файл 2.bat должен иметь следующий вид:

```
SET a=1  
ECHO a=%a% (SET a=2 ECHO a=!a!)
```

ВНИМАНИЕ! Приведенный материал необходим для правильной работы команды цикла FOR и будет использован в командных файлах!

Приостановка выполнения командных файлов

Для того, чтобы вручную **прервать выполнение** запущенного bat- файла, нужно нажать клавиши <Ctrl>+<C> или <Ctrl>+<Break>. Однако часто бывает необходимо программно приостановить выполнение командного файла в определенной строке с выдачей запроса на нажатие любой клавиши. Это делается с помощью команды PAUSE. Перед запуском этой команды полезно с помощью команды **ECHO** информировать пользователя о действиях, которые он должен произвести. Например:

```
ECHO Вставьте дискету в дисковод A: и нажмите любую клавишу PAUSE
```

Команду **PAUSE** обязательно нужно использовать при выполнении потенциально опасных действий (удаление файлов, форматирование дисков и т.п.). Например,

```
ECHO Сейчас будут удалены все файлы в C:\Мои документы! ECHO Для  
отмены нажмите Ctrl-C
```

```
PAUSE  
DEL "C:\Мои документы\*.*)"
```

Вызов внешних командных файлов

Из одного командного файла можно вызвать другой, просто указав его имя. Например:

```
@ECHO OFF CLS  
REM Вывод списка log-файлов DIR C:\*.log  
:: Передача выполнения файлу f.bat f.bat  
COPY A:\*.* C:\ PAUSE
```

Однако в этом случае после выполнения вызванного файла управление в вызывающий файл не передается, то есть в приведенном примере команда

```
COPY A:\*.* C:\
```

(и все следующие за ней команды) никогда не будет выполнена.

Для того, чтобы вызвать внешний командный файл с последующим возвратом в первоначальный файл, нужно использовать специальную команду CALL файл Например:

```
@ECHO OFF CLS  
:: Вывод списка log-файлов DIR C:\*.log  
:: Передача выполнения файлу f.bat CALL f.bat  
COPY A:\*.* C:\ PAUSE
```

В этом случае после завершения работы файла f.bat управление вернется в

первоначальный файл на строку, следующую за командой **CALL** (в нашем примере это команда **COPY A:*.* C:**).

Операторы перехода **GOTO** и вызова **CALL**

Командный файл может содержать метки и команды **GOTO** перехода к этим меткам. Любая строка, начинающаяся с двоеточия :, воспринимается при обработке командного файла как метка. Имя метки задается набором символов, следующих за двоеточием до первого пробела или конца строки.

Пример 3. Пусть имеется командный файл следующего содержания:

```
@ECHO OFF
```

```
COPY %1 %2
```

```
GOTO Label1
```

```
ECHO Эта строка никогда не выполнится
```

```
:Label1
```

```
:: Продолжение выполнения DIR %2
```

После того, как в этом файле мы доходим до команды **GOTO Label1** его выполнение продолжается со строки

```
:: Продолжение выполнения
```

В команде перехода внутри файла **GOTO** можно задавать в качестве метки перехода строку **:EOF**, которая передает управление в конец текущего пакетного файла (это позволяет легко выйти из пакетного файла без определения каких-либо меток в самом его конце).

Для перехода к метке внутри текущего командного файла кроме команды **GOTO** можно использовать и рассмотренную выше команду **CALL**:

```
CALL :метка аргументы
```

При вызове такой команды создается новый контекст текущего пакетного файла с заданными аргументами, и управление передается на инструкцию, расположенную сразу после метки. Для выхода из такого пакетного файла необходимо два раза достичь его конца. Первый выход возвращает управление на инструкцию, расположенную сразу после строки **CALL**, а второй выход завершает выполнение пакетного файла. Например, если запустить с параметром **"Копия-1"** командный файл следующего содержания:

```
@ECHO OFF ECHO %1
```

```
CALL :2 Копия-2
```

```
:2
```

```
ECHO %1
```

то на экран выведутся три строки:

```
Копия-1 Копия-2 Копия-1
```

Таким образом, подобное использование команды **CALL** похоже на вызов подпрограмм в языках высокого уровня.

Оператор проверки условия **IF**

С помощью команды **IF ... ELSE** (ключевое слово **ELSE** может отсутствовать) в пакетных файлах можно выполнять обработку условий

нескольких типов. При этом если заданное после **IF** условие принимает истинное значение, система выполняет следующую за условием команду (или несколько команд, заключенных в круглые скобки), в противном случае выполняется команда (или несколько команд в скобках), следующие за ключевым словом **ELSE**.

Проверка значения переменной

Первый тип условия используется обычно для проверки значения переменной. Для этого применяются два варианта синтаксиса команды **IF**: **IF [NOT] строка1==строка2 команда1 [ELSE команда2]**

(квадратные скобки указывают на необязательность заключенных в них параметров) или

IF [/I] [NOT] строка1 оператор_сравнения строка2 команда

Рассмотрим сначала первый вариант. Условие **строка1==строка2** (здесь необходимо писать именно два знака равенства – как и в программах на C/C++) считается истинным при точном совпадении обеих строк. Параметр **NOT** указывает на то, что заданная команда выполняется лишь в том случае, когда сравниваемые строки не совпадают.

Для *группировки команд* могут использоваться круглые скобки. Иногда использование круглых скобок необходимо для правильной работы команды **if...else** – например для вывода на экран наибольшего из двух параметров, с которыми запущен командный файл, следует использовать оператор

if %1 GTR %2 (echo %1) else (echo %2)

Строки могут быть литеральными или представлять собой значения переменных (например, **%1** или **%TEMP%**). Кавычки для литеральных строк **не требуются**. Например,

IF %1==%2 ECHO Параметры совпадают!

IF %1==windows ECHO значение первого параметра - windows

Отметим, что при сравнении строк, заданных переменными, следует проявлять определенную осторожность. Дело в том, что значение переменной может оказаться пустой строкой, и тогда может возникнуть ситуация, при которой выполнение командного файла аварийно завершится. Например, если вы не определили с помощью команды **SET** переменную **MyVar**, а в файле имеется условный оператор типа

IF %MyVar%==C:\ ECHO Ура!!!, то в процессе выполнения вместо

%MyVar% подставится пустая строка и возникнет синтаксическая ошибка.

Такая же ситуация может возникнуть, если одна из сравниваемых строк является значением параметра командной строки, так как этот параметр может быть не указан при запуске командного файла. Поэтому при сравнении строк нужно приписывать к ним в начале какой-нибудь символ, например:

IF -%MyVar%==-C:\ ECHO Ура!!!

С помощью команд **IF** и **SHIFT** можно в цикле обрабатывать все параметры командной строки файла, даже не зная заранее их количества. Например, следующий командный файл (назовем его **primer.bat**) выводит на

экран имя запускаемого файла и все параметры командной строки: @ECHO OFF
 ECHO Выполняется файл: %0 ECHO.
 ECHO Файл запущен со следующими параметрами...
 :: Начало цикла
 :BegLoop
 IF -%1== GOTO ExitLoop ECHO %1
 :: Сдвиг параметров SHIFT
 :: Переход на начало цикла GOTO BegLoop
 :ExitLoop
 :: Выход из цикла ECHO.
 ECHO Все.

Если запустить primer.bat с четырьмя параметрами:
 primer.bat A B C D то в результате выполнения на экран выведется
 следующая информация:

Выполняется файл: primer.bat
 Файл запущен со следующими параметрами: A
 B C D
 Все.

Рассмотрим теперь оператор **IF** в следующем виде: IF [/I] строка1
 оператор_сравнения строка2 команда

Синтаксис и значение **операторов_сравнения** представлены в (табл.
 3).

Таблица 3 - Операторы сравнения в IF

Оператор	Значение
EQL	Равно
NEQ	Не равно
LSS	Меньше
LEQ	Меньше или равно
GTR	Больше
GEQ	Больше или равно

Пример 4. использования операторов сравнения:

@ECHO OFF CLS

IF -%1 EQL –Вася ECHO Привет, Вася!

IF -%1 NEQ –Вася ECHO Привет, но Вы не Вася!

Ключ **/I**, если он указан, задает сравнение текстовых строк **без учета регистра**. Ключ **/I** можно также использовать и в форме **строка1==строка2**

команды **IF**. Например, условие
IF /I DOS==dos ... будет истинным.

Проверка существования заданного файла

Второй способ использования команды **IF** — это проверка существования заданного файла. Синтаксис для этого случая имеет вид:

IF [NOT] EXIST файл команда1 [ELSE команда2]

Условие считается истинным, если указанный файл существует. Кавычки для имени файла не требуются. Приведем пример командного файла, в котором с помощью такого варианта команды IF проверяется наличие файла, указанного в качестве параметра командной строки. @ECHO OFF

IF -%1==- GOTO NoFileSpecified
IF NOT EXIST %1 GOTO FileNotExist

:: Вывод сообщения о найденном файле ECHO Файл '%1' найден.
GOTO :EOF

:NoFileSpecified

:: Файл запущен без параметров

ECHO В командной строке не указано имя файла. GOTO :EOF

:FileNotExist

:: Параметр командной строки задан, но файл не найден ECHO Файл '%1'
не найден.

Проверка наличия переменной среды

Аналогично файлам команда **IF** позволяет проверить наличие в системе определенной переменной среды:

IF DEFINED переменная команда1 [ELSE команда2]

Здесь условие **DEFINED** применяется подобно условию **EXISTS** наличия заданного файла, но принимает в качестве аргумента имя переменной среды и возвращает истинное значение, если эта переменная определена. Например:

@ECHO OFF CLS

IF DEFINED MyVar GOTO :VarExists ECHO Переменная MyVar не
определена GOTO :EOF

:VarExists

ECHO Переменная MyVar определена, ECHO ее значение равно %MyVar%

Проверка кода завершения предыдущей команды

Еще один способ использования команды **IF** — это проверка кода завершения (кода выхода) предыдущей команды. Синтаксис для **IF** в этом случае имеет следующий вид:

IF [NOT] ERRORLEVEL число команда1 [ELSE команда2]

Здесь условие считается истинным, если последняя запущенная команда

или программа завершилась с кодом возврата, равным либо превышающим указанное число.

Рассмотрим командный файл, который копирует файл my.txt на диск C: без вывода на экран сообщений о копировании, а в случае возникновения какой-либо ошибки выдает предупреждение:

```
@ECHO OFF
XCOPY my.txt C:\ > NUL
:: Проверка кода завершения копирования IF ERRORLEVEL 1 GOTO
ErrOccurred ECHO Копирование выполнено без ошибок. GOTO :EOF
:ErrOccurred
ECHO При выполнении команды XCOPY возникла ошибка!
```

В операторе **IF ERRORLEVEL ...** можно также применять операторы сравнения чисел, приведенные в табл. 2. Например:

```
IF ERRORLEVEL LEQ 1 GOTO Case1
```

Замечание. Иногда более удобным для работы с кодами завершения программ может оказаться использование переменной **%ERRORLEVEL%**. (строковое представление текущего значения кода ошибки **ERRORLEVEL**).

Организация циклов

В командных файлах для организации циклов используются несколько разновидностей оператора **FOR**, которые обеспечивают следующие функции:

- выполнение заданной команды для всех элементов указанного множества;
- выполнение заданной команды для всех подходящих имен файлов;
- выполнение заданной команды для всех подходящих имен каталогов;
- выполнение заданной команды для определенного каталога, а также всех его подкаталогов;
- получение последовательности чисел с заданными началом, концом и шагом приращения;
- чтение и обработка строк из текстового файла; обработка строк вывода определенной команды.

Цикл **FOR ... IN ... DO ...**

Самый простой вариант синтаксиса команды **FOR** для командных файлов имеет следующий вид:

```
FOR %%переменная IN (множество) DO команда [параметры]
```

Внимание!

Перед названием переменной должны стоять именно два знака процента (**%%**), а не один, как это было при использовании команды **FOR** непосредственно из командной строки!

Пример 5. Если в командном файле заданы строки **@ECHO OFF**
FOR %%i IN (Раз, Два, Три) DO ECHO %%i

то в результате его выполнения на экран будет выведено следующее: Раз
Два Три

Параметр **множество** в команде **FOR** задает одну или более текстовых строк, разделенных запятыми, которые необходимо обработать с помощью заданной команды. Скобки здесь **обязательны**. Параметр **команда [параметры]** задает команду, выполняемую для каждого элемента множества, при этом вложенность команд **FOR** на одной строке **не допускается**. Если в строке, входящей во множество, используется запятая, то значение этой строки нужно заключить в кавычки. Например, в результате выполнения файла с командами

```
@ECHO OFF
```

```
FOR %%i IN ("Раз,Два",Три) DO ECHO %%i
```

на экран будет выведено Раз,Два

Три

Параметр **%%переменная** представляет подставляемую переменную (счетчик цикла), причем здесь могут использоваться только имена переменных, **состоящие из одной буквы**. При выполнении команда **FOR** заменяет подставляемую переменную текстом каждой строки в заданном множестве, пока команда, стоящая после ключевого слова **DO**, не обработает все такие строки.

Замечание. Чтобы избежать путаницы с параметрами командного файла %0 — %9, для переменных следует использовать любые символы кроме 0 — 9.

Параметр **множество** в команде **FOR** может также представлять одну или несколько групп файлов. Например, чтобы вывести в файл список всех файлов с расширениями txt и prn, находящихся в каталоге C:\TEXT, без использования команды **DIR**, можно использовать командный файл следующего содержания:

```
@ECHO OFF
```

```
FOR %%f IN (C:\TEXT\*.txt C:\TEXT\*.prn) DO ECHO %%f >> list.txt
```

При таком использовании команды **FOR** процесс обработки продолжается, пока не обработаются все файлы (или группы файлов), указанные во множестве.

Цикл **FOR /D ... IN ... DO ...**

Следующий вариант команды **FOR** реализуется с помощью ключа **/D** (directory – каталог):

```
FOR /D %%переменная IN (набор) DO команда [параметры]
```

В случае, если набор содержит подстановочные знаки, то команда выполняется для всех подходящих имен каталогов, а не имен файлов. Скажем, выполнив следующий командный файл:

```
@ECHO OFF CLS
```

```
FOR /D %%f IN (C:\*.*) DO ECHO %%f
```

мы получим список всех каталогов на диске C:, например: C:\Arc

C:\CYR C:\MSCAN

C:\Program Files C:\TEMP C:\WINNT

Цикл FOR /R ... IN ... DO ...

С помощью ключа **/R** можно задать **рекурсию** в команде **FOR**: **FOR /R** [[диск:]путь] %переменная IN (набор)

DO команда [параметры]

В этом случае заданная команда выполняется для каталога [диск:] путь, а также для **всех подкаталогов** этого пути. Если после ключа **R** не указано имя каталога, то выполнение команды начинается с текущего каталога.

Пример 6. Для распечатки всех файлов с расширением txt в текущем каталоге и всех его подкаталогах можно использовать следующий пакетный файл:

```
@ECHO OFF CLS
```

```
FOR /R %%f IN (*.txt) DO PRINT %%f
```

Если вместо набора указана только точка (.), то команда проверяет все подкаталоги текущего каталога. Например, если мы находимся в каталоге C:\TEXT с двумя подкаталогами BOOKS и ARTICLES, то в результате выполнения файла:

```
@ECHO OFF CLS
```

```
FOR /R %%f IN (.) DO ECHO %%f
```

на экран выведутся три строки:

```
C:\TEXT\ C:\TEXT\BOOKS\ C:\TEXT\ARTICLES\
```

Цикл FOR /L ... IN ... DO ...

Ключ **/L** позволяет реализовать с помощью команды **FOR** цикл со счетчиком, в этом случае синтаксис имеет следующий вид:

FOR /L %переменная IN (начало, шаг, конец) **DO** команда [параметры] Здесь заданная после ключевого слова **IN** тройка (начало, шаг,

конец) задает последовательность чисел с заданными началом, концом и шагом приращения. Например, тройка (1, 1, 5) порождает последовательность (1 2 3 4 5), а тройка (5, -1, 1) - последовательность (5 4 3 2 1). Например, в результате выполнения следующего командного файла:

```
@ECHO OFF
```

```
CLS
```

```
FOR /L %%f IN (1,1,5) DO ECHO %%f
```

переменная цикла **%%f** получит значения от 1 до 5, и на экран будут выведены пять чисел:

```
1
```

```
2
```

```
3
```

```
4
```

```
5
```

Числа, получаемые в результате выполнения цикла **FOR /L**, можно использовать в **арифметических вычислениях**. Рассмотрим командный файл my.bat следующего содержания:

```
@ECHO OFF CLS
```

```
FOR /L %%f IN (1,1,5) DO CALL :2 %%f GOTO :EOF
:2
SET /A M=10*%%1 ECHO 10*%%1=%M%
```

В третьей строке в цикле происходит вызов нового контекста файла `my.bat` с текущим значением переменной цикла `%%f` в качестве параметра командной строки, причем управление передается на метку `:2` (см. описание `CALL` в разделе "Изменения в командах перехода"). В шестой строке переменная цикла умножается на десять, и результат записывается в переменную `M`. Таким образом, в результате выполнения этого файла выведется следующая информация:

```
10*1=10
10*2=20
10*3=30
10*4=40
10*5=50
```

Цикл FOR /F ... IN ... DO ...

Самые широкие возможности имеет команда `FOR` с ключом `/F`: `FOR /F ["ключи"] %переменная IN (набор) DO команда [параметры]`

Здесь параметр `набор` содержит имена одного или нескольких файлов, которые по очереди открываются, читаются и обрабатываются. Обработка состоит в чтении файла, разбиении его на отдельные строки текста и выделении из каждой строки заданного числа подстрок. Затем найденная подстрока используется в качестве значения переменной при выполнении основного тела цикла (заданной команды).

По умолчанию ключ `/F` выделяет из каждой строки файла первое слово, очищенное от окружающих его пробелов. Пустые строки в файле пропускаются. Необязательный параметр "ключи" служит для переопределения заданных по умолчанию правил обработки строк. Ключи представляют собой заключенную в кавычки строку, содержащую приведенные в (табл. 4) ключевые слова:

Таблица 4 - Ключи в команде FOR /F

Ключ	Описание
<code>EOL=C</code>	Определение символа комментариев в начале строки (допускается задание только одного символа)
<code>SKIP=N</code>	Число пропускаемых при обработке строк в начале файла
<code>DELIMS=XXX</code>	Определение набора разделителей для замены заданных по умолчанию пробела и знака табуляции
<code>TOKENS=X, Y, M-N</code>	Определение номеров подстрок, выделяемых из каждой строки файла и передаваемых для выполнения в тело цикла

При использовании ключа `TOKENS=X, Y, M-N` создаются дополнительные

переменные. Формат **M-N** представляет собой диапазон подстрок с номерами от **M** до **N**. Если последний символ в строке **TOKENS=** является звездочкой, то создается дополнительная переменная, значением которой будет весь текст, оставшийся в строке после обработки последней подстроки.

Разберем применение этой команды на примере пакетного файла parser.bat, который производит разбор файла myfile.txt:

```
@ECHO OFF
IF NOT EXIST myfile.txt GOTO :NoFile
FOR /F "EOL=; TOKENS=2,3* DELIMS=, " %%i IN (myfile.txt) DO @ECHO
%%i %%j %%k GOTO :EOF
:NOFile
ECHO Не найден файл myfile.txt!
```

Здесь во второй строке производится проверка наличия файла myfile.txt; в случае отсутствия этого файла выводится предупреждающее сообщение. Команда **FOR** в третьей строке обрабатывает файл myfile.txt следующим образом:

Пропускаются все строки, которые начинаются с символа точки с запятой (**EOL=;**).

Вторая и третья подстроки из каждой строки передаются в тело цикла, причем подстроки разделяются пробелами (по умолчанию) и/или запятыми (**DELIMS=,**).

В теле цикла переменная **%%i** используется для второй подстроки, **%%j** — для третьей, а **%%k** получает все оставшиеся подстроки после третьей.

Замечание. Имена переменных **i, j, k** должны следовать в алфавитном порядке.

В нашем примере переменная **%%i** явно описана в инструкции **FOR**, а переменные **%%j** и **%%k** описываются **неявно** с помощью ключа **TOKENS=**. Например, если в файле myfile.txt были записаны следующие три строки: AAA BBVB CCCC,GGGG DDDD

```
EEEE,JJJ KKKK
;TTTT LLLL MMMMM
```

то в результате выполнения пакетного файла parser.bat на экран выведется следующее:

```
BBVB CCCC GGGG DDDD JJJ KKKK
```

Замечание. Ключ **TOKENS=** позволяет извлечь из одной строки файла до 26 подстрок, поэтому запрещено использовать имена переменных, начинающиеся не с букв английского алфавита (a–z). Следует помнить, что имена переменных **FOR** являются **глобальными**, поэтому одновременно не может быть активно более 26 переменных.

Команда **FOR /F** также позволяет обработать отдельную строку. Для этого следует ввести нужную строку в кавычках вместо набора имен файлов в скобках. Строка будет обработана так, как будто она взята из файла. Например, файл следующего содержания:

```
@ECHO OFF
```

```
FOR /F "EOL=; TOKENS=2,3* DELIMS=, " %%i IN ("AA CC BB,GG DD")
DO @ECHO %%i %%j %%k
```

при своем выполнении напечатает CC BB GG DD

Вместо явного задания строки для разбора
можно пользоваться переменными среды, например:

```
@ECHO OFF
```

```
SET M=AAA BBBB BBBB,GGGG ДДДД FOR /F "EOL=; TOKENS=2,3*
DELIMS=,
" %%i IN ("%M%") DO @ECHO %%i %%j %%k
```

Наконец, команда **FOR /F** позволяет обработать **строку вывода другой команды**. Для этого следует вместо набора имен файлов в скобках ввести строку вызова команды в апострофах (не в кавычках!). Строка передается для выполнения интерпретатору команд cmd.exe, а вывод этой команды записывается в память и обрабатывается так, как будто строка вывода взята из файла. Например, следующий командный файл:

```
@ECHO OFF CLS
```

```
ECHO Имена переменных среды:
```

```
ECHO.
```

```
FOR /F "DELIMS==" %%i IN ('SET') DO ECHO %%i
```

выведет перечень имен всех переменных
среды, определенных в настоящее время в системе.

В цикле **FOR** допускается применение тех же синтаксических конструкций (операторов), что и для заменяемых параметров – (табл 5).

Таблица 5 - Операторы для переменных команды FOR

Операторы	Описание
%%~Fi	Переменная %i расширяется до полного имени файла
%%~Di	Из переменной %i выделяется только имя диска
%%~Pi	Из переменной %i выделяется только путь к файлу
%%~Ni	Из переменной %i выделяется только имя файла
%%~Xi	Из переменной %i выделяется расширение имени файла
%%~Si	Значение операторов N и X для переменной %i изменяется так, что они работают с кратким именем файла
%%~Zi	Определяется длина (размер) файла с указанным именем

Замечание. Если планируется использовать расширения подстановки значений в команде **FOR**, то следует внимательно подбирать имена переменных, чтобы они не пересекались с обозначениями формата.

Например, если мы находимся в каталоге C:\Program Files\Far и запустим командный файл следующего содержания:

```
@ECHO OFF CLS
```

```
FOR %%i IN (*.txt) DO ECHO %%~Fi
```

то на экран выведутся полные имена всех файлов с расширением txt:

C:\Program Files\Far\Contacts.txt

C:\Program Files\Far\ReadMe.txt C:\Program Files\Far\register.txt C:\Program Files\Far\WhatsNew.txt

Вычисление суммарной длины всех файлов в заданном подкаталоге
@ECHO OFF

SETLOCAL ENABLEDELAYEDEXPANSION

Set /a Size = 0

For %%I in (%1*.*) do set /a Size= Size + %%~zI Echo %Size%

Циклы и связывание времени выполнения для переменных

Как и в рассмотренном выше примере с составными выражениями, при обработке переменных среды внутри цикла могут возникать труднообъяснимые ошибки, связанные с ранними связыванием переменных. Рассмотрим пример. Пусть имеется командный файл следующего содержания:

SET a=

FOR %%i IN (Раз,Два,Три) DO SET a=%a%%i ECHO a=%a%

В результате его выполнения на экран будет выведена строка "a=Три", то есть фактически команда

FOR %%i IN (Раз,Два,Три) DO SET a=%a%%i

равносильна команде

FOR %%i IN (Раз,Два,Три) DO SET a=%%i

Для исправления ситуации нужно, как и в случае с составными выражениями, вместо знаков процента (%) использовать восклицательные знаки и предварительно включить режим связывания времени выполнения командой **SETLOCAL ENABLEDELAYEDEXPANSION**. Таким образом, наш пример следует переписать следующим образом:

SETLOCAL ENABLEDELAYEDEXPANSION SET a=

FOR %%i IN (One,Two,Three) DO SET a=!a!%%i ECHO a=%a%

В этом случае на экран будет выведена строка "a=OneTwoThree".

Команда Findstr и ее использование в цикле

Назначение команды - поиск строк в текстовых файлах.

FINDSTR [/B] [/E] [/L] [/R] [/S] [/I] [/X] [/V] [/N] [/M] [/O] [/P] [/F:файл]
[/C:строка] [/G:файл] [/D:список_папок] [/A:цвета] [/OFF[LINE]] строки
[[диск:][путь]имя_файла[...]]

/L-Поиск строк дословно.

/R-Поиск строк как регулярных выражений.

/S-Поиск файлов в текущей папке и всех ее подпапках.

/I-Определяет, что поиск будет вестись без учета регистра.

/X-Печатает строки, которые совпадают точно.

/V-Печатает строки, не содержащие совпадений с искомыми.

/N-Печатает номер строки, в которой найдено совпадение, и ее содержимое.
 /M-Печатает только имя файла, в которой найдено совпадение.
 /O-Печатает найденный строки через пустую строку.
 /P-Пропускает строки, содержащие непечатаемые символы.
 /F:файл-Читает список файлов из заданного файла (/ для консоли).
 /C:строка-Использует заданную строку как искомую фразу поиска.
 /D:список_папок-Поиск в списке папок (разделяются точкой с запятой).
 строка Искомый текст.
 [диск:][путь]имя_файла - задает имя файла или файлов.

Использовать пробелы для разделения нескольких искомых строк, если аргумент не имеет префикса /C. Например, 'FINDSTR "Привет мир" a.b' ищет "Привет" или "мир" в файле a.b, а команда 'FINDSTR /C:"Привет мир" a.b' ищет строку

"Привет мир" в файле a.b.

Краткая сводка по синтаксису регулярных выражений:

. Любой символ.
 * Повтор: ноль или более вхождений предыдущего символа или класса
 ^ Позиция в строке: начало строки
 \$ Позиция в строке: конец строки
 [класс] Класс символов: любой единичный символ из множества [^класс]
 Обратный класс символов: любой единичный символ из дополнения
 [x-y] Диапазон: любые символы из указанного диапазона
 \x Служебный символ: символьное обозначение служебного символа x
 \<xyzПозиция в слове: в начале слова xyz\> Позиция в слове: в конце слова

Пример командного файла для поиска в файле num.txt по образцу строк, в которых присутствует хотя бы одна двоичная цифра.

```
@echo off set /a kol=0
```

```
for /f %%b in ('findstr /rc:"[0-1]" num.txt') do set /a kol=kol+1 echo %kol%
```

МЕТОДИКА ВЫПОЛНЕНИЯ

1. Ознакомиться с теоретическими сведениями.
2. Запустить интерпретатор командной строки
3. Увеличить размер окна интерпретатора и задать цвет фона и цвет шрифта (рекомендуется синий фон и белый шрифт).
4. Создать список фамилий студентов группы, используя пример 1. Отсортировать список в алфавитном порядке и сохранить его в новом файле.

Замечание 1. При создании текстового файла интерпретатор командной строки использует кодировку **кириллица (DOS)**. Поэтому рекомендуется переназначить вывод в файл с расширением **.txt**, а для просмотра содержимого файла использовать Internet Explorer, указав вид кодировки кириллица (DOS).

Замечание 2. Интерпретатор хранит историю введенных команд в буфере (размером 50 строк). Для просмотра содержимого буфера используйте клавиши клавиатуры СТРЕЛКА ВВЕРХ и СТРЕЛКА ВНИЗ. Полученную команду можно отредактировать и выполнить снова.

5. Создать текстовый файл, содержащий справочные сведения по командам DIR, COPY и XCOPY.

6. Вывести содержимое указанного в табл.6 каталога по указанному формату на экран и в файл.

Таблица 6 – Варианты заданий для студентов по списку

Номера бригад	Имя каталога	Что выводить	Сортировать по	Атрибуты фай-лов и каталогов
1, 6	%Windows%	Только файлы	По размеру	Системный
2, 7	%Windows%	Файлы и подкаталоги	По дате	Скрытый
3, 8	%Windows%	Только подкаталоги	Именам	Только чтение
4, 9	%Windows% и все подкаталоги	Только файлы bmp	По размеру	Только чтение
5, 10	%Windows% и все подкаталоги	Только файлы jpg	Именам	Любые

7. Скопировать все имеющиеся в каталоге Windows растровые графические файлы в каталог WinGrafika на диске C:. Если диск C: недоступен, использовать любой другой доступный диск. / Скопировать все имеющиеся в каталоге Windows исполняемые файлы в каталог WinEx на диске C:. Если диск C: недоступен, использовать любой другой доступный диск.

Замечание. Для подготовки текстов командных файлов рекомендуется использовать блокнот (Notepad). При этом следует избегать использования в выводимых на экран результатах работы командного файла букв русского алфавита.

8. Разработать и выполнить командные файлы (КФ), выполняющие следующие функции:

1) Вывод на экран имен всех файлов с указанным расширением, находящихся в каталоге, имя которого задается при запуске командного файла первым параметром. Расширение файлов задается вторым параметром.

2) Среди введенных с клавиатуры целых чисел (использовать SET /P) найти наибольшее и наименьшее. Признаком конца ввода – знак -.

3) В заданном каталоге и его подкаталогах найти общее количество подкаталогов. На экран вывести только требуемый результат.

4) В каталогах, имена которых заданы первым и вторым параметрами командного файла, найти и вывести на экран имена файлов (расширения могут быть любые), присутствующие как в первом, так и во втором каталоге. Следует использовать только один оператор FOR.

5) Вычисление и вывод на экран значения факториала целого числа, задаваемого при запуске КФ. Предусмотреть проверку заданного значения и при задании отрицательного значения или значения, превышающего максимально возможную величину, выводить соответствующие сообщения. Для проверки правильности вычислений использовать калькулятор.

9. Разработать и выполнить КФ в соответствии с таблицей 7 (индивидуальные задания для студентов), номер задания соответствует списочному номеру студента.

Таблица 7 - Индивидуальные задания для студентов

Номер	Действия, выполняемые КФ
1	1. Подсчет количества целых чисел в текстовом файле. Считать, что слова в файле записаны в формате ОДНО СЛОВО В СТРОКЕ. Слово – это целое число (состоящее из десятичных цифр) или последовательность букв латинского алфавита (начинающаяся с буквы). Имя файла задается первым параметром КФ. 2. Вывод на экран списка файлов, хранящихся в указанном первым параметром каталоге и созданных в первом полугодии (месяцы 1-6) года, указанного вторым параметром КФ.
2	1. В каталоге, указанном первым параметром КФ, (и его подкаталогах) найти файл наибольшего размера с расширением, указанным вторым параметром КФ. 2. В каталоге, указанном первым параметром КФ, (и его подкаталогах) найти ТРИ файла самого большого размера. Вывести имена файлов, их размеры и даты создания
3	1. Разбиение текстового файла, имя которого задано первым параметром КФ, на три файла с именами 1.txt, 2.txt и 3.txt. Количество строк в каждом из этих файлов задано вторым, третьим и четвертым параметрами КФ. Проверить наличие указанного исходного файла и вывести сообщение о его отсутствии, проверить наличие остальных параметров и их значения на допустимость 2. В каталоге, указанном первым параметром КФ, (и его подкаталогах) найти суммарный объем файлов, имеющих расширение, указанное вторым параметром КФ.

4	1. Удаление из каталога, заданного первым параметром, файлов, которые присутствуют и в каталоге, указанным вторым параметром. Предусмотреть запрос пользователю на подтверждение удаления. 2. В каталоге, указанном первым параметром КФ, и его подкаталогах, найти файлы, созданные во второй половине рабочего дня (после 14 часов) и скопировать их в отдельный подкаталог.
5	1. Нахождение суммарного объема файлов с атрибутом system, хранящихся в каталоге, имя которого задано первым параметром КФ. 2. Проверить наличие файла Numb.txt в каталоге, указанном первым параметром КФ. Прочитать целые числа из файла, найти среди них простые и вывести результаты на экран. Считать, что все числа не превышают значения 2500.
6	1. Поиск на диске C: (или любом доступном диске) файла с заданным именем. Если файл не найден – вывод сообщения. Если файл найден – открыть его для редактирования. 2. Проверка наличия на диске в каталоге, указанном первым параметром КФ, файла FNames.txt, содержащего список имен файлов и подкаталогов. Если он есть – проверка наличия перечисленных в списке файлов и вывод имен файлов/каталогов, отсутствующих в каталоге. Если файла FNames нет, создание его и запись имен файлов и подкаталогов.
7	1. Вывод списка DLL (хранящихся на доступном диске), созданных до 12.2021 размером до 12000 байтов. 2. Проверка наличия на диске в каталоге, указанном первым параметром КФ, файла Numbers.txt, содержащего 2 столбца целых чисел, столбцы располагаются с позиций 2 и 20 и отделены пробелами. Если файла нет – вывод сообщения. Если файл есть, создать новый файл, содержащий три столбца, в третий поместить сумму чисел из двух первых столбцов.
8	1. Просмотр содержимого каталога, указанного первым параметром КФ. Необходимо: 1. создать подкаталоги с именами EXE, TXT, CMD, DOC и OTHER. 2. В каждый подкаталог скопировать файлы с соответствующими расширениями. 3. Пустые подкаталоги удалить. 2. В каталоге, указанном первым параметром КФ, (и его подкаталогах) найти файлы наибольшего и наименьшего размеров. Вывести имена файлов, их размеры и даты создания.

9	1. Проверка наличия трех текстовых файлов на диске и объединения их в один файл. 2. Подсчет количества вещественных чисел и целых чисел в текстовом файле. Вещественные и целые числа подсчитать отдельно. Считать, что слова в файле записаны в формате ОДНО СЛОВО В СТРОКЕ. Слово – это целое число (состоящее из десятичных цифр) или последовательность букв латинского алфавита (начинающаяся с буквы) или последовательность десятичных цифр с точкой (.) внутри строки. Имя файла задается первым параметром КФ.
10	1. Подсчет количества слов в текстовом файле, содержащем целые числа и слова. Считать, что слова в файле записаны в формате ОДНО СЛОВО В СТРОКЕ. Число – это целое число (состоящее из десятичных цифр). Слово - последовательность букв латинского алфавита (начинающаяся с буквы). Имя файла задается первым параметром КФ. 2. Просмотр содержимого каталога, указанного первым параметром КФ. Необходимо: создать подкаталоги с именами 1, 2, ..., 12. В каждый подкаталог скопировать файлы, созданные в соответствующие месяцы. Пустые подкаталоги удалить.
11	1. Подсчет количества строк в текстовом файле, имя которого задано первым параметром КФ. Проверить наличие указанного файла и вывести сообщение о его отсутствии. 2. С помощью команды DIR вывести на экран имена файлов, находящихся в каталоге, имя которого задано первым параметром КФ. Второй и остальные параметры задают расширения файлов, имена которых выводить не следует. Рекомендуется с помощью ATTRIB присвоить некоторым файлам атрибут СКРЫТЫЙ – такие файлы DIR не показывает.
12	1. Поиск текстового файла по его содержимому. Считать, что слова в текстовых файлах записаны в формате ОДНО СЛОВО В СТРОКЕ. Искомое слово задается первым параметром КФ. 2. Проверка наличия трех текстовых файлов на диске и объединения их в один файл.
13	1. Вывод на экран аргументов, с которыми КФ был запущен. Число аргументов от 4 до 11. При неверном числе аргументов ничего не выполнять, сообщить об ошибке. 2. Поиск и вывод на экран минимального и максимального значения аргумента КФ. Предполагается, что все аргументы КФ – целые положительные числа.

Примечание. Для решения задач 1, 9 и 10 рекомендуется использовать команду Findstr

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Различие между внутренними и внешними командами. Примеры внешних и внутренних команд.
2. Структура команды интерпретатора.
3. Получение информации о конкретной команде.
4. Групповые символы (шаблоны) и их использование.
5. Перенаправление ввода/вывода и конвейеризация команд.
6. Условное выполнение и группировка команд.
7. Назначение символов `&`, `&&`, `||` и `()`.
8. Команды для работы с файловой системой – названия и возможности.
9. Достоинства и недостатки команд `COPY` и `XCOPY`.
10. Назначение команды `ECHO` и примеры ее использования.
11. Команда `DIR` и ее возможности.
12. В какой кодировке интерпретатор выводит информацию и как получить читаемую твердую копию?
13. Вывод сообщений и дублирование команд.
14. Использование параметров командной строки.
15. Переменные среды, получение и изменение их значений.
16. Операции со строковыми и числовыми переменными.
17. Проверка существования заданного файла и наличия переменной среды.
18. Выполнение заданной команды для всех элементов указанного множества.
19. Выполнение заданной команды для всех подходящих имен файлов.
20. Выполнение заданной команды для всех подходящих имен каталогов.
21. Выполнение заданной команды для определенного каталога, а также всех его подкаталогов.
22. Получение последовательности чисел с заданными началом, концом и шагом приращения.
23. Чтение и обработка строк из текстового файла.
24. Команда `Findstr`. Назначение. Ключи. Использование регулярных выражений в команде. Задание и использование класса цифр и класса букв через диапазон.
25. Операторы перехода и вызова.
26. Какое минимальное количество строк (включая `@echo off`) должен иметь командный файл, выводящий на экран минимальное значения двух числовых аргументов?
27. Какое минимальное количество строк (включая `@echo off`) должен иметь командный файл, выводящий на экран минимальное значения трех числовых аргументов?